Sintaxis del lenguaje_II - Tipos de datos Primitivos

Cada valor almacenado tiene un tipo de dato que describe que tipo de cosa es, un número, un texto, … El compilador puede deducir los tipos de las variables basándose en sus asignaciones iniciales.

Rust es un lenguaje ***estáticamente tipado***, lo que significa que debe conocer los tipos de todas las variables **en tiempo de compilación**. Dependiendo del tipo de variable, el compilador optimizará la memoria necesaria.

⇒ Tipos de datos primitivos

- Son fundamentales y básicos
- Están integrados en el lenguaje (no necesitan bibliotecas ni módulos)
- Tienen soporte directo del compilador. El compilador los conoce y optimiza su uso
- No se pueden descomponer, no se construyen con datos más simples

⇒ Tipos de datos no primitivos

Los tipos de datos no primitivos en Rust se construyen a partir de los tipos primitivos o son definidos por el usuario.

Estos tipos ofrecen estructuras más complejas y flexibilidad en la programación.

- **Primitivos**
  - **escalares**: Entero, flotante, booleano, carácter
  - **compuestos**: Array, tupla y Slice (secuencias)

- **No primitivos** (se construyen con no primitivos o definidos por el usuario)
  - String
    - String Slices (&str)
    - String (String)
  - Vectores (Vect<T>)
  - Estructuras (struc)
  - Enumeraciones (enums)
  - Tipos genéricos y parametrizados



Tipos de datos Escalares

Contienen un solo valor (Enteros con o sin signo, punto flotante, booleanos y caracteres)

Tipos Enteros

Con signo (i)*

Tipo	Bits	Mínimo	Máximo
i8	8	-128	127
i16	16	-32 768	32 767
i32	32	-2 147 483 648	2 147 483 647
i64	64	-9 223 372 036 854 775 808	9 223 372 036 854 775 807
i128	128	-170 141 183 460 469 231 731 687 303 715 884 105 728	170 141 183 460 469 231 731 687 303 715 884 105 727
isize	32/64 (según arquitectura)	-2^{31} o -2^{63}	$2^{31}-1$ o $2^{63}-1$

Sin signo (u)*

Tipo	Bits	Mínimo	Máximo
u8	8	0	255
u16	16	0	65 535
u32	32	0	4 294 967 295
u64	64	0	18 446 744 073 709 551 615
u128	128	0	340 282 366 920 938 463 463 374 607 431 768 211 455
usize	32/64 (según arquitectura)	0	$2^{32}-1$ o $2^{64}-1$

```
    println!("Integer / Enteros");
    let num_a :i128 = 167;  // numero entero de 128 bits
    let num_b :i64 = 16560; // numero entero de 64 bits
    let num_c :i32 = 167;   // numero entero de 32 bits
    let num_d :i16 = 16700; // numero entero de 16 bits
    let num_e :i8 = 120;   // numero entero de 8 bits

    // declarcion corta
    let num_f = 12; //rust asigna el tipo de dato según el valor asignado
```

Rust nos ofrece las constantes asociadas **MAX** y **MIN** para saber los rangos de cada tipo.

```
// maximo y minimo valor
    println!("---------------------------------");
    println!(" i8 min valor {}.", i8::MIN);
    println!(" i8 max valor {}.", i8::MAX);
    println!(" i16 min valor {}.", i16::MIN);
    println!(" i16 has the max value of {}.", i16::MAX);
```

```
// números enteros positivos (unsigned)
    println!("---------------------------------");
    println!(" u8 min valor {}.", u8::MIN);
    println!(" u8 max valor {}.", u8::MAX);
    println!(" u16 min valor {}.", u16::MIN);
    println!(" u16 has the max value of {}.", u16::MAX);
```

```
 // números enteros tipos arch (arquitectura)
    println!("---------------------------------");
    println!(" isize min valor {}.", isize::MIN);
    println!(" isize max valor {}.", isize::MAX);
    println!(" usize min valor {}.", usize::MIN);
    println!(" usize max valor {}.", usize::MAX);
```

Con esta función extraída de la documentación de Rust se puede consultar el tipo de una variable

```
// funcion para saber el tipo de dato
fn type_of<T>(_: &T) -> &'static str {
    std::any::type_name::<T>()
}
 println!("El tipo de dato es: {} \n", type_of(&num_a));
```

Guiones para lectura de números (son un adorno, Rust no los interpreta)

```
//rust permite usar guiones bajos para separar los números y hacerlos más legibles
let num_g = 34_456_000;

//Los guiones se pueden poner en cualquier lugar porque no son significativos
let num_x = 3_4_456_0_00;
```

Ejemplos de algunos métodos con enteros:

Método	Descripción	Ejemplo	Resultado
abs()	Valor absoluto	(-5i32).abs()	5
signum()	Signo (-1, 0, 1)	(-7i32).signum()	-1
is_positive()	¿Es mayor que 0?	5i32.is_positive()	true
is_negative()	¿Es menor que 0?	(-5i32).is_negative()	true
pow(exp)	Potencia	2i32.pow(10)	1024
isqrt()	Raíz cuadrada entera	17u32.isqrt()	4
abs_diff(otro)	Diferencia absoluta	3u32.abs_diff(10)	7
min(otro)	El menor	3.min(7)	3
max(otro)	El mayor	3.max(7)	7
clamp(min, max)	Limita a un rango	15.clamp(0, 10)	10

Coma flotante

Type	Precisión max	Rango
f32	6 - 9 dígitos de precisión	±3.40282347e+38
f64	15 – 17 dígitos de precisión	±1.7976931348623157e+308

**se corresponden con los tipos float y double de otros lenguajes

Operaciones matemáticas:

```
let x: f32 = 3.14; // Declaración de un valor f32
let y: f64 = 3.14; // Declaración de un valor f64
// let suma  =  x + y;
/*  Error: Rust no hace conversiones explicitas entre tipos de datos diferentes,
por lo que no se puede sumar un f32 con un f64 directamente.
Tendríamos que usar el mismo tipo de variable para poder hacer operaciones aritméticas entre ellas.
o usar un casting para convertir un tipo de dato a otro  */

let suma = x + y as f32; // Convertimos y a f32 para poder sumar
let resta = x as f64 - y;

//convertir solo una vez
let x64 = f64::from(x);
let multiplicacion = x64 * y;
let division = x64 / y;
```

Funciones más usadas con float:

- **`abs()`** → Devuelve el valor absoluto de un número.
- **`ceil()`** → Redondea hacia arriba al número entero más cercano.
- **`floor()`** → Redondea hacia abajo al número entero más cercano.
- **`round()`** → Redondea al número entero más cercano.
- **`sqrt()`** → Calcula la raíz cuadrada del número.

```
    let pi: f64 = 3.14159265358979323384;
    println!("The current value of pi is {pi}");

    println!("{}", pi.floor());
    println!("{}", pi.ceil());
    println!("{}", pi.round());
```

Presentación de decimales en float

Se puede establecer el número de decimales que queremos visualizar

```
    let pi: f64 = 3.14159265358979323384;
    println!("The current value of pi is {:.4}", pi);
    println!("The current value of pi is {pi:.4}");

// formateo decimal para float
    println!("1) Formateo Flotante (presición)");
    let num_flot : f64 = 12560.7895;
    println!("a) {} b) {:.2} c) {:.4} d) {:.8} ", num_flot,num_flot,num_flot,
num_flot );
```

Caracteres

Tipo **char** → Representan un solo carácter Unicode de 4 bytes (32 bits).

Son inmutables y pueden representar letras, dígitos, símbolos, caracteres de escape y emojis.

Pagina para copiar emojis → https://emojipicker.com/

```
    let letra_dni = 'B';
    let cara: char = '😀';
```

```rust
let mensaje: &str = "Hola 🌍"; // &str es referencia a cadena de tipo primitivo
let saludo = "Hola\nMundo"; // \n es un salto de línea
println!(
    "letra_dni: {}, cara: {}, mensaje: {} y saludo: {}",
    letra_dni, cara, mensaje, saludo
);
let familia = "👨‍👩‍👧";// varios code points unidos con ZWJ
println!("{}", familia.chars().count()); // 5
```

Caracteres de escape en Rust → Se escriben con una barra invertida `\` dentro de literales `char` y cadenas.

Escape	Significado	Ejemplo
`\n`	Salto de línea	`"línea1\nlínea2"`
`\r`	Retorno de carro	`"texto\r\n"` (estilo Windows)
`\t`	Tabulador	`"a\tb"`
`\\`	Barra invertida	`"C:\\carpeta"`
`\"`	Comilla doble	`"dijo \"hola\""`
`\'`	Comilla simple	`'\''`
`\0`	Carácter nulo	`'\0'`

```rust
println!("Nombre:\tAna\nEdad:\t30");
println!("Dijo: \"Hola\"");
println!("Ruta: C:\\Users\\Ana");
println!("Emoji: \u{1F680}");
println!("Cruda: {}", r"sin \n escape"); //con r no interpreta la \ de escape
println!("Llaves: {{ y }}");   // Llaves escapan con otra llave  {{
```

Funciones más usadas

- **is_alphabetic()** → Devuelve true si el carácter es una letra alfabética.
- **is_numeric()** → Devuelve true si el carácter es un dígito numérico.
- **to_uppercase()** → Convierte el carácter a mayúsculas.
- **to_lowercase()** → Convierte el carácter a minúsculas.
- **is_whitespace()** → Devuelve true si el carácter es un espacio en blanco.
- **is_ascii()** → Devuelve true si el carácter es un carácter ASCII de 7 bits.

```rust
let letra_dni = 'B';
let cara: char = '😀';
println!("cara: {cara}");
println!("{} {}", letra_dni.is_alphabetic(), cara.is_alphabetic());
println!("{} {}", letra_dni.is_uppercase(), cara.is_uppercase());
println!("{} {}", letra_dni.is_lowercase(), cara.is_lowercase());
```

Booleanos

Tipo **bool** → Representa un valor lógico que puede ser true o false.

```rust
let t = true;
let f: bool = false;

let edad: i32 = -40;
let es_joven = edad <= 35;
println!("{es_joven}");
println!("{} {}", edad.is_positive(), edad.is_negative());
```

El operador **!** invierte el valor de un booleano.

El resultado de comparaciones con los operadores **&&** y **||** devolverá un booleano

```rust
let boleto_comprado = true;
let avion_a_tiempo = true;
let llegare_al_evento = boleto_comprado && avion_a_tiempo;
println!("Es {} que llegaré como se esperaba", llegare_al_evento);
```

Primitivos Compuestos

Compuesto es que “está formado por varias partes”, por lo que un tipo compuesto puede agrupar varios valores en un solo tipo.

Arrays

- Colección de datos homogéneos (mismo tipo) **de tamaño fijo** durante todo el programa.
- El array se almacena en posiciones continuas de memoria y el acceso a los elementos se realiza mediante índices (el primero es 0)
- Son inmutables, si queremos cambiar algún valor En el programa hay que declararlo mutable.

```
   let a: [i32; 5] = [1, 2, 3, 4, 5];
   let numeros = [4, 8, 15, 16, 23, 42];
   let frutas : [&str; 3] = ["Manzana", "Melón", "Pera"];
//se establece el tamaño, Rust necesita saberlo por gestión de memoria

    // declaracion corta
    let arreglo_str = ["Rust", "Go", "Python", "Java"];
    let arreglo_int = [1992, 2015, 2020, 2023];
    let arreglo_float = [12.50, 33.00, 12.89, 99.95];

    // declaracion larga
    let arr_str2: [&str; 3] = ["Ene", "Feb", "Mar"];
    let arr_int2: [i32; 2] = [15, 30];

    println!("Longitud = {}", arreglo_str.len());
    println!("Contenido Pos 1 = {}", arreglo_str[1]);
```

Se pueden declarar arrays vacíos, pero hay que indicar el tipo de dato. También se pueden rellenar los elementos con un mismo valor.

```
    let arreglo_vacio: [i32; 0] = [];
    println!("arreglo_vacio {:?}", arreglo_vacio);

    let ceros = [0; 4]; // 4 columnas, todo en 0
```

Los arrays no se pueden visualizar con un **display** de tipo cadena dentro de las llaves {} de la macro prinln!() como otros tipos de variables

```
 println!("arreglo_str {}", arreglo_str);
```

Si intentamos hacerlo, Rust nos dará un error indicando que no implementa Display.

```
error[E0277]: `[&str; 4]` doesn't implement `std::fmt::Display`
  --> src\bin\05_arrays.rs:16:28
   |
16 | println!("arreglo_str {}", arreglo_str);
   |                        --   ^^^^^^^^^^^ `[&str; 4]` cannot be formatted with the default formatter
   |                        |
   |                        required by this formatting parameter
   |
   = help: the trait `std::fmt::Display` is not implemented for `[&str; 4]`
   = note: in format strings you may be able to use `{:?}` (or {:#?} for pretty-print) instead
```

Y también muestra la solución, utilizar el símbolo llamado “especificador de formato” **:** (que se usa, por ejemplo, para los decimales de un float) seguido de **?**

```
// mostrar arreglos
  println!("arreglo_str {:?}", arreglo_str);
  println!("arreglo_float {:?}", arreglo_float);
  println!("arreglo_int {:?}", arreglo_int);
  println!("arr_str2 {:?}", arr_str2);
  println!("arr_int2 {:?}", arr_int2);
```

y si queremos interpolarlo:

```
    println!("arreglo_str {arr_str2:?}" );
```

También existe la opción con hashtag que visualiza los datos con salto de línea entre ellos

```
    println!("arreglo_str {arr_str2:#?}" );
```

Existe una macro **dbg!(…)** que permite visualizar contenidos como operaciones matemáticas directas, contenidos de arrays, … que es usado por los desarrolladores para comprobaciones (no se debe usar en producción)

```
  dbg!(2 + 2);
  dbg!(&arreglo_str);
```

Arrays multidimensionales

Los arrays multidimensionales permiten almacenar los datos en forma de matriz (filas y columnas), el tamaño es fijo y es inmutable (si queremos cambiar valores hay que declararla como mutable)

```
// Array 2D de 3 filas y 4 columnas
let matriz: [[i32; 4]; 3] = [   //“3 arrays de 4 enteros i32”
        [1, 2, 3, 4],
        [5, 6, 7, 8],
        [9, 10, 11, 12],
];

// Acceder a un elemento (fila 1, columna 2)
println!("Elemento [1][2]: {}", matriz[1][2]);  // 7

// Recorrer el array
for fila in matriz.iter() {
        for valor in fila.iter() {
                print!("{} ", valor);
        }
        println!();
}
//Declarar y rellenar con cero todos los elementos
let ceros = [[0; 4]; 3]; // 3 filas x 4 columnas, todo en 0
```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

Tupla

- Una tupla **puede almacenar varios valores de distintos tipos** lo que permite modelar mejor objeto del mundo real. (nombre, sueldo, departamento)
- Tienen una longitud fija
- Son inmutables

```
    let tupla = (42, "Rust", 3.14);
    let cantidad = tup.0;
    let tup: (i32, f64, u8) = (500, 6.4, 1);
    println!("tupla {:?}", tupla);
```

Se pueden extraer los valores de la tupla en variables.

```
    let tup = (500, 6.4, 1);
    let (x, y, z) = tup;  //desectructuración
    println!("The value of y is: {y}");
    println!("El segundo valor es: {}", tup.1);
```

Una tupla vacía, cero elementos, se llama **unit** () que representa “ningún valor significativo” y se escribe ()

```
fn saludar() {
    println!("Hola"); //una instrucción que termina en ; devuelve unit()
}

// Equivale a:
fn saludar_explicito() -> () {
    println!("Hola");
}
```